

המרכז האקדמי לב

**הפקולטה להנדסה ומדעי המחשב
החוג להנדסת חשמל ואלקטרוניקה**

פרויקט גמר באלקטרוניקה

פיתוח מערכת מבוססת FPGA להתראת נפילה לחולי דמנציה

**קליטת תמונה בזמן אמת ממצלמת OV7670 והצגתה דרך
כרטיס Artix-7 על מנת לאפשר עיבוד תמונה לזיהוי
עמידה ממושכת לחולי דמנציה.**

**מגיש: יוסי ברים
מנחה: מר ערן שלום**

תודות

תודה למר ערן שלום
על הליווי ועל כל הסבלנות וההשקעה בכל מהלך הפרויקט, ממש ממש.

למר אורי שטרואו היקר
על העזרה בפרויקט והכלים שנתן לי בכל מהלך הלימודים.

תודה רבה לד"ר דוד מרסלו
על כל הידע שקיבלתי בכל מהלך התואר ועל הליווי בפרויקט.

לפרופסור בני מילגרום, ראש החוג
על ההתחשבות והעזרה בכל מהלך התואר.

לד"ר יוסף גולובצ'וב, רכז החוג
על ההתחשבות ועל הידע שנתן לי בתואר.

תודה ענקית לאבא ולאמא שלי המדהימים
על הכל.

לחברים המדהימים שלי,
לאחותי,
ולאלעד השותף הנפלא.

תודה

תקציר

בעקבות הגידול באוכלוסייה המבוגרת והעלייה בשיעור המתמודדים עם דמנציה, עולה הצורך במערכות ניטור בטיחותיות בסביבה הביתית. תופעה מסוכנת היא עמידה ממושכת ללא מודעות לעייפות, המובילה לנפילות ופציעות. פרויקט זה מציע פתרון טכנולוגי מתקדם המבוסס על עיבוד וידאו חומרתי, המייתר את הצורך בחיישנים לבישים ומבטיח תגובה בזמן אמת.

בליבת הפרויקט פותחה פלטפורמת חומרה מקבילית ויציבה על גבי רכיב FPGA ממשפחת Artix-7. המערכת מנהלת מסלול נתונים שלם: החל מקליטת תמונה גולמית ממצלמת OV7670, דרך זיכרון BRAM בתצורת Dual-Port, ועד ליציאה של בקר VGA ייעודי הכולל ממיר אותות מבוסס רשת נגדים (DAC) להצגת הווידאו.

הסיפור המרכזי ועיקר המיקוד בעבודה זו הוא אימות התכנן (Verification) והוכחת האמינות של מסלול הנתונים מהזיכרון למסך. תהליך זה בוצע בשני רבדים הנדסיים מחמירים: תחילה, בוצע אימות לוגי מלא ברמת קוד המקור (Source) בסביבת ModelSim, אשר אישר עמידה קפדנית בתזמוני התקן של פרוטוקול ה-VGA. לאחר מכן, בוצע דיבוג ואימות חומרה פיזי ברמת הסיליקון (Post-Silicon) באמצעות כלי ה-ILA של Vivado. הבדיקות המקיפות אישרו זרימת נתונים חלקה, חסינות לגליצים וסנכרון מושלם מול המסך.

הצלחת שלבים אלו מספקת תשתית חומרתית איתנה ואמינה לעיבוד הווידאו (שאותה ידגים שותפי אלעד אסבג), כקינח וכיוון פיתוח עתידי להפחתת התראות שווא, הודגם תוכנית מודל בינה מלאכותית (Edge AI) קליל לזיהוי אנושי, מתוך הצעה עתידית להטמיעו על גבי מעבד פנימי (MicroBlaze) בארכיטקטורת חומרה-תוכנה משולבת (HW/SW Co-Design).

Abstract

Due to the growth of the elderly population and the rising rate of individuals coping with dementia, there is an increasing need for home safety monitoring systems. A dangerous phenomenon is prolonged standing without awareness of fatigue, which leads to falls and injuries. This project proposes an advanced technological solution based on hardware-level video processing, eliminating the need for wearable sensors and ensuring a real-time response.

At the core of the project, a parallel and stable hardware platform was developed on an Artix-7 FPGA. The system manages a complete data path: starting from capturing raw images from an OV7670 camera, through a Dual-Port BRAM, and up to the output of a dedicated VGA controller that includes a resistor-network-based Digital-to-Analog Converter (DAC) for video display.

The main narrative and primary focus of this work is the design verification and reliability proof of the data path from the memory to the screen. This process was executed in two rigorous engineering stages: first, a full logical verification at the source code level was performed in the ModelSim environment, which confirmed strict adherence to the timing standards of the VGA protocol. Subsequently, post-silicon physical hardware debugging and verification were performed using Vivado's Integrated Logic Analyzer (ILA). The comprehensive testing confirmed a smooth data flow, glitch immunity, and perfect synchronization with the display.

The success of these stages provides a robust and reliable hardware foundation for the video processing that will be demonstrated by my partner, Elad Asbag. As a final addition and a future development direction to reduce false alarms, a lightweight Edge AI human detection model was demonstrated in software, with the future ambition of embedding it on an internal processor (MicroBlaze) within a Hardware/Software Co-Design architecture.

תוכן העניינים

9	1 מבוא
9	1.1 רקע ומוטיבציה
9	1.2 מטרת הפרויקט
9	1.3 גישה לפתרון וחלוקת עבודה
11	2 רקע תיאורטי וסקירת ספרות
11	2.1 מבוא לפרוטוקול VGA וקונספט תותח האלקטרוניס (CRT)
11	2.2 סריקה קווית (Raster Scan) ומנגנון ה-Retrace
12	2.3 תזמוני המסך: מהפיזיקה למימוש הלוגי (RTL)
14	3 פיתוח ושיטות
14	3.1 ארכיטקטורת המערכת וזרימת הנתונים
14	3.2 מימוש בקר התצוגה (VGA Controller)
15	3.2.1 עיבוד הפיקסל והמרה לאנלוגית (DAC)
18	3.3 מבט-על: חלוקת המערכת, ארכיטקטורת זיכרון ותחומי שרון
18	3.3.1 מבט-על על המערכת וזרימת הנתונים
19	3.3.2 מחולל השעונים (Clock Generator)
19	3.3.3 זיכרון התמונה (BRAM Frame Buffer)
20	3.3.4 ממשק הזיכרון וסנכרון נתונים למסך
22	4 תוצאות
22	4.1 אימות לוגי ModelSim
22	4.1.1 אימות מסלול הנתונים והסנכרון: VGA Controller ↔ BRAM
22	4.1.2 אימות חוקיות הפרוטוקול - מבט-על על מחזור שורה (T_s)
23	4.1.3 אימות דופק הסנכרון האופקי (T_{pw})
24	4.1.4 אימות מרווח ההתייבבות בתחילת שורה (T_{bp} - Back Porch)
24	4.1.5 אימות האזור הפעיל (T_{disp} - Active Video)
25	4.1.6 אימות סיום השורה ומרווח הביטחון (T_{fp} - Front Porch)
25	4.2 אימות חומרה פיזי ILA
25	4.2.1 מעבר מתיאוריה לסיליקון
26	4.2.2 עלות הדיבוג וניתוח Trade-offs בשימוש ב-ILA
27	4.2.3 אימות מסלול הנתונים בזמן אמת (Post-Silicon Validation)
29	5 דיונים
29	5.1 ניתוח בעיות סנכרון
סנכרון	5.1 ניתוח בעיות
29	
29	5.1 ניתוח בעיות סנכרון
29	5.2 חומרה ייעודית (FPGA) ושילוב מודלי בינה מלאכותית (Edge AI)

(VHDL) תצוגה בקר - המקור קוד א': נספח א'	34
(Testbench) סימולציה אימות - המקור קוד ב': נספח ב'	39

רשימת האיורים

11	מבנה פנימי של מסך CRT והמחשת מנגנון תותח האלקטרונים	1
12	המחשת מסלול הסריקה הקווית (Raster Scan) ותהליכי ההחזרה (Retrace) האופקית והאנכית	2
14	תרשים מלבנים של ארכיטקטורת המערכת וזרימת הנתונים	3
15	תרשים תזמונים של המונים האנכיים והאופקיים בבקר ה-VGA	4
16	תרשים זרימה של עיבוד הפיקסל: מזיכרון ה-BRAM, דרך מנגנון הרישום והסנכרון, ועד לניתוב לערוצי ה-RGB	5
17	סכמה חשמלית של רשת הנגדים (DAC) ומחבר ה-HD-DB15	6
18	תרשים חשמלי המציג את סולם הנגדים בכרטיס (משמאל) אל מול נגד הסיומת במסך (מימין)	7
18	מבט-על של המערכת: חלוקה לליבות עיבוד, תקשורת קליטה (אדום) ותקשורת תצוגה (כחול)	8
19	ממשק הקינפוג של ה-Clocking Wizard ב-Vivado להפקת שעוני התצוגה והמצלמה	9
20	תרשים בלוקים של זיכרון ה-BRAM בתצורת Dual-Port המאפשרת הפרדת תחומי שרון	10
21	קטע מקוד ה-VHDL המציג את תרגום הקואורדינטות בזמן אמת ואת מנגנון ה-Pipeline Register	11
22	סימולציית ModelSim המאמת את מסלול הנתונים הפנימי: ייצוב הנתון הנשלף מה-BRAM על ידי רגיסטר הדגימה, והעברתו לערוצי ה-RGB	12
23	מבט-על של סימולציית מחזור שורה שלם (T_s): אימות ספירת המונה האופקי מ-0 עד 799	13
24	התמקדות בדופק הסנכרון האופקי (T_{pw}): המונה רץ מ-0 עד 95, פולס ה-Sync מופעל, וערוצי הצבע מושתקים	14
25	אימות שלב ה-Back Porch: השתקת ערוצי הצבע לאורך 144 המחזורים הראשונים של השורה	15
26	אימות האזור הפעיל (T_{disp}): מנייה של 640 מחזורים (783-144) ופעילות רציפה של אפיק הכתובת	16
27	אימות ה-Front Porch: 16 מחזורי שרון (784-799) של השתקת נתונים בסוף השורה	17
28	המחשת מגבלת הדיבוג החיצוני והצורך בפתיחת "הקופסה השחורה" באמצעות שתלי ILA	18
29	מפת המשאבים על גבי הסיליקון: מימין המערכת ללא ILA, ומשמאל תוספת החומרה עבור מערכת הדיבוג	19
30	השוואת דוחות Utilization ב-Vivado המציגים את תקורת ה-ILA	20
31	דגימת ILA מתוך הסיליקון המאמת את זרימת הנתונים בזמן אמת	21
32	הוכחת היתכנות על גבי Raspberry Pi: זיהוי נפילה בזמן אמת ומעקב אחר מצבי ההתראה	22
33	הצעת הארכיטקטורה העתידית	23

רשימת הטבלאות

1	פרמטרי תזמון VGA - מהצורך הפיזיקלי ועד למימוש בחומרה	13
---	--	----

1 מבוא

1.1 רקע ומוטיבציה

אוכלוסיית העולם המבוגרת גדלה בקצב מהיר, ועימה עולה שיעור האנשים המתמודדים עם בעיות זיכרון וירידה ביכולות הקוגניטיביות. אחת הבעיות המשמעותיות באוכלוסייה זו היא הקושי בהתנהלות בטוחה בסביבה הביתית. תופעה מסוכנת נפוצה היא עמידה ממושכת ללא שימת לב לעייפות המצטברת, מה שעלול להוביל לניסיון התיישבות פתאומי ללא ודאות שקיים כיסא מאחור, וכתוצאה מכך – לנפילות ופציעות חמורות. במקרים רבים, ההשלכות הפיזיות של נפילות אלו פוגעות באיכות החיים בצורה אנושה.

קיים אפוא צורך מהותי בפיתוח מערכות ניטור אוטומטיות, אשר לא ידרשו מהמטופל לשאת חיישנים לבישים (Wearables), אלא יתבססו על ניתוח המרחב באמצעות מצלמות ועיבוד תמונה מתקדם בשולי הרשת (Edge Computing).

1.2 מטרת הפרויקט

מטרתו המרכזית של הפרויקט היא לפתח פלטפורמת חומרה יציבה, מקבילית ויעילה, המנהלת את מסלול הנתונים השלם (Data Path) – החל מקליטת התמונה ממצלמת ה-OV7670, דרך אגירתה בזיכרון, ועד להצגתה הרציפה. הדגש המרכזי בפיתוח המערכת על גבי רכיב FPGA הוא היכולת לעבד את הנתונים הוויזואליים בזמן אמת, יכולת זו מהווה יתרון קריטי וחסר תחליף עבור מערכת ניטור רפואית הנדרשת להגיב באופן מיידי לאירועי חירום, כדוגמת נפילת מטופל. הפלטפורמה נועדה להוות תשתית חומרתית איתנה, שעליה ניתן יהיה להריץ מערכת התראות אוטונומית מתקדמת. עבודה זו מתמקדת בתכנון מסלול הנתונים לתצוגה, פתרון אתגרי הסנכרון הפנימיים, וביצוע אימות לוגי ופיזי מחמיר.

1.3 גישה לפתרון וחלוקת עבודה

פיתוח המערכת מתבסס על קוד תשתיתי שפותח בעבר על ידי סטודנט קודם. כמו כן, אימות התכן בחלק של קליטת הנתונים – מהמצלמה ועד לזיכרון התמונה (BRAM) – בוצע והושלם על ידי סטודנט נוסף בעבר.

העבודה הנוכחית מתמקדת בהשלמת מסלול הנתונים, הוספת שכבת זיהוי התנועה, והתוויית הארכיטקטורה לזיהוי חכם. תהליך זה התבצע כרונולוגית ופונקציונלית בשלושה שלבים עיקריים:

אימות תכן תצוגה וחומרה (יוסי ברים - חלק ראשון):

שלב זה מתמקד באימות מחמיר של מסלול הנתונים מהזיכרון ועד להצגה על המסך. התהליך מוצג דרך מימוש ובחינה של פרוטוקול ה-VGA, ומבוצע בשני רבדים: תחילה אימות ברמת הקוד (Source) והלוגיקה באמצעות כלי הסימולציה ModelSim, ולאחר מכן ולידציה של החומרה הפיזית עצמה בשלב ה-Post-Silicon באמצעות כלי ה-ILA מבית Vivado.

עיבוד תמונה גולמי וגילוי תנועה (אלעד אסבג):

לאחר הבטחת יציבות מסלול הנתונים, שולבה לוגיקה לזיהוי שינויים במרחב הפיקסלים (Frame Differencing). שכבה זו מתריעה על כל תנועה במרחב.

שדרוג ארכיטקטוני לבינה מלאכותית (יוסי ברים - חלק שני):

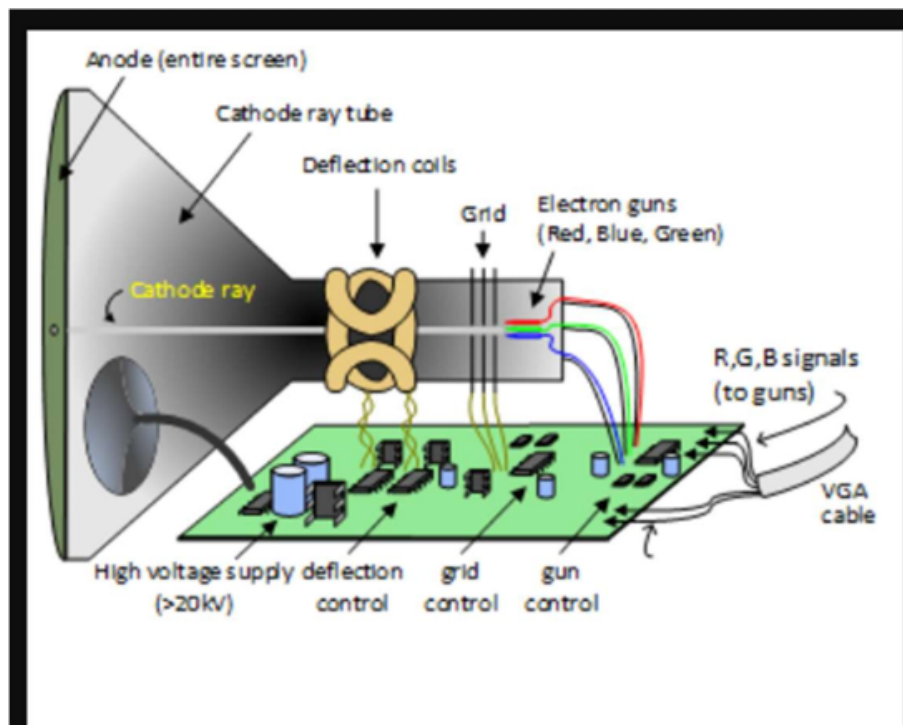
כדי לתת מענה לאתגר התראות השווא, מוצעת ארכיטקטורה המשלבת מודל בינה מלאכותית (AI) שתפקידו לברור ולזהות ספציפית דמות אדם מתוך סך השינויים שעליהם המערכת כבר יודעת להתריע. הקונספט הומחש והוכח מעשית (PoC) בסביבת תוכנה על גבי Pi. Raspberry ברמת ה-FPGA, יישום זה מוצג כהצעה ארכיטקטונית עתידית (תכנון HW/SW משולב עם מעבד פנימי).

2 רקע תיאורטי וסקירת ספרות

2.1 מבוא לפרוטוקול VGA וקונספט תותח האלקטרוניס (CRT)

טכנולוגיית ה-VGA פותחה במקור עבור מסכי שופרת קרן קתודית (CRT - Cathode Ray Tube), אשר פעולתם מבוססת על פיזיקה אנלוגית. מנגנון התצוגה פועל באמצעות "תותח אלקטרוניס" היורה קרן פיזית על גבי מסך המצופה בזרחן, כאשר עוצמת הזרם החשמלי היא זו שקובעת את רמת ההארה של כל פיקסל.

נקודה קריטית להבנת תזמוני הפרוטוקול היא המורשת האנלוגית שלו: אף על פי שהמסכים המודרניים כיום מבוססים על טכנולוגיות דיגיטליות (כגון LCD), פרוטוקול ה-VGA עדיין מחייב שידור של אותות השהייה. אותות אלו נועדו היסטורית לאפשר לקרן האלקטרוניס הפיזית את זמן התנועה הנדרש כדי לחזור לתחילת שורה או לתחילת מסך (Retrace), ולכן גם בקרים מודרניים ב-FPGA נדרשים לממש זמנים אלו במדויק על מנת שהמסך יסתכרך כראוי.



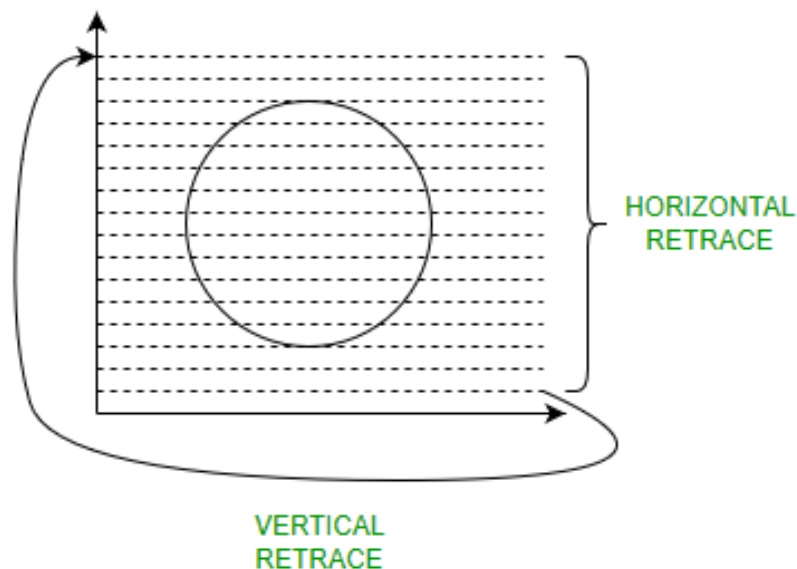
איור 1: מבנה פנימי של מסך CRT והמחשת מנגנון תותח האלקטרוניס

2.2 סריקה קווית (Raster Scan) ומנגנון ה-Retrace

הצגת התמונה על גבי המסך מתבצעת בשיטת סריקה קווית (Raster Scan). בשיטה זו, ציור התמונה מבוסס על קרן הסורקת את מערך הפיקסלים באופן שיטתי ומחזורי: משמאל לימין עבור כל שורה בנפרד, ומלמעלה למטה עבור הפריים כולו.

מאחר שתנועת הקרן חייבת להיות רציפה, נוצר מסלול סריקה דמוי "זיגזג" לאורך המסך. עובדה זו מנציחה אילוץ הנדסי ופיזיקלי מרכזי בפרוטוקול: בכל פעם שהקרן מגיעה לקצה אזור התצוגה – בין אם בסוף שורה (קצה אופקי) ובין אם בתחתית המסך בסיום הפריים (קצה אנכי) – נדרש פסק זמן מוגדר כדי להחזיר אותה לנקודת ההתחלה.

פסק זמן זה מכונה החזרה (Retrace) אופקית או אנכית, בהתאמה. בזמנים אלו חובה להבטיח שלא משודר מידע ויזואלי למסך (Blanking), והם מנוהלים במערכת באמצעות אותות הסנכרון כדי לוודא יציבות של התמונה בזמן שהקרן עושה את דרכה חזרה.



איור 2: המחשת מסלול הסריקה הקווית (Raster Scan) ותהליכי ההחזרה (Retrace) האופקית והאנכית

2.3 תזמוני המסך: מהפיזיקה למימוש הלוגי (RTL)

כדי ליישם את סריקת המסך וההחזרות (Retrace) שנידונו לעיל, הפרוטוקול מגדיר ארבעה מצבי תזמון עיקריים עבור כל שורה (אופקי) ועבור כל פריים (אנכי). הטבלה הבאה מתארת את הפרמטרים האופקיים (עבור רזולוציה של 640×480), את הצורך הפיזיקלי המקורי במסכי ה-CRT, וכיצד אילוץ זה מתורגם הלכה למעשה למימוש הדיגיטלי (RTL) במערכת ה-FPGA שפותחה:

פרמטר	מחזורי שעות	הצורך הפיזיקלי המקורי (CRT)	המימוש שלנו (RTL)
Active Display	640	הקרן מציירת את הפיקסלים הפיזיים על המסך משמאל לימין.	שליפת נתונים מה-BRAM ושידור צבע (RGB) פעיל.
Front Porch	16	מרווח זמן לקרן להתייבב בסוף השורה לפני שחוזרת אחורה.	כיבוי מיידי של ה-RGB ל-'0000' למניעת מריחות.
Sync Pulse	96	מכת מתח (טריגר) שמאלצת את הקרן לקפוץ חזרה שמאלה.	הורדת האות הפיזי HSYNC ל-'0' למשך 96 שעות.
Back Porch	48	המתנה לייצוב הקרן בתחילת השורה החדשה בצד שמאל.	המשך השהיית ה-RGB על '0000' עד הגעה לפיקסל הראשון.

טבלה 1: פרמטרי תזמון VGA - מהצורך הפיזיקלי ועד למימוש בחומרה

3 פיתוח ושיטות

3.1 ארכיטקטורת המערכת וזרימת הנתונים

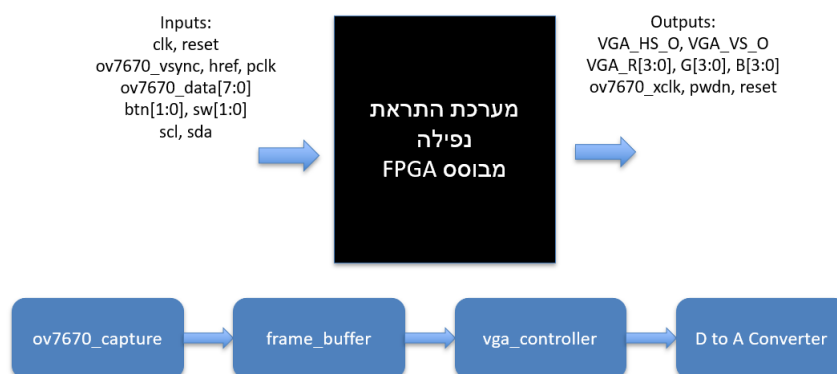
זרימת הנתונים במערכת בנויה כשרשרת עיבוד רציפה (Pipeline) המורכבת מארבעה מודולים עיקריים המשורשרים זה לזה:

1. **קליטת נתונים (OV7670_capture):** דגימת נתוני התמונה הגולמיים מחומרת המצלמה בזמן אמת.

2. **אוגר תמונה (Frame Buffer):** אגירת הנתונים בזיכרון פנימי מובנה מסוג BRAM לגישור על פערי תדרים.

3. **בקר תצוגה (VGA Controller):** שליפת הנתונים מהזיכרון וייצור אותות הבקרה למסך בסנכרון מושלם.

4. **ממיר אותות (D to A Converter):** המרת האותות הדיגיטליים לרמות מתח אנלוגיות התואמות לתקן התצוגה.



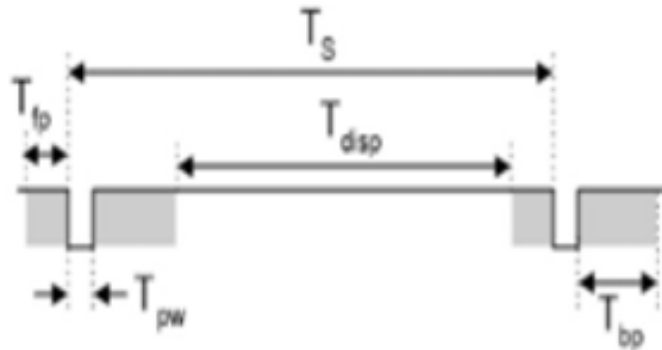
איור 3: תרשים מלבנים של ארכיטקטורת המערכת וזרימת הנתונים

3.2 מימוש בקר התצוגה (VGA Controller)

בקר ה-VGA מיושם בחומרה באמצעות שני מונים סינכרוניים: מונה אופקי ומונה אנכי. מחזור שורה שלם מורכב מ-800 פיקסלים (חיבור של 640 פיקסלים פעילים, 16 מחזורי Front Porch, 96 מחזורי Sync Pulse, ו-48 מחזורי Back Porch). לפיכך, המונה האופקי (hsync_reg) סופר החל מ-0 ועד 799 ומתאפס.

המערכת פועלת בתדר שעון פיקסל של 25 MHz (זמן מחזור של 40 ns), המבטיח קצב רענון תקני של 60 Hz. מילון האותות המרכזי של הבקר כולל את pxl_clk (שעון הפיקסלים), / hsync_reg

vsync_reg (מוני הקואורדינטות), ו-in_display_area (דגל לוגי לזיהוי אזור פעיל). כאשר המונה נמצא מחוץ לתחום התצוגה הפעילה (מצבי ה-Blanking), הלוגיקה מאלצת את אותות הצבע (RGB) לערך '0000' למניעת מריחות צבע בעת החזרת הקרן.

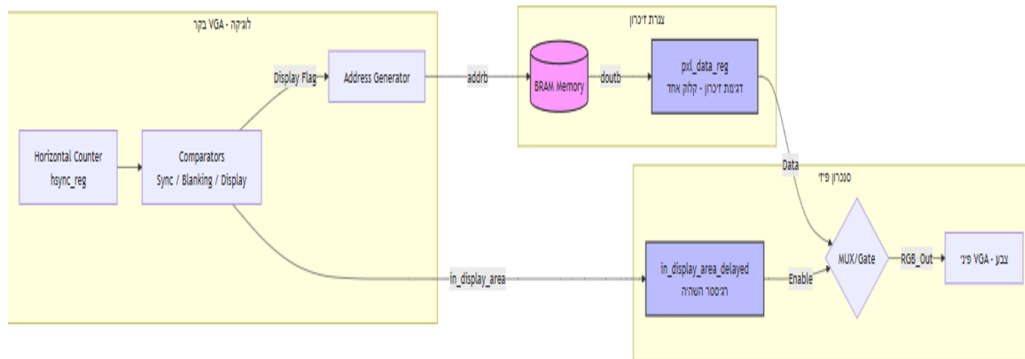


Symbol	Parameter	Vertical Sync			Horiz. Sync	
		Time	Clocks	Lines	Time	Clks
T_S	Sync pulse	16.7ms	416,800	521	32 us	800
T_{disp}	Display time	15.36ms	384,000	480	25.6 us	640
T_{pw}	Pulse width	64 us	1,600	2	3.84 us	96
T_{fw}	Front porch	320 us	8,000	10	640 ns	16
T_{bp}	Back porch	928 us	23,200	29	1.92 us	48

איור 4: תרשים תזמונים של המונים האנכיים והאופקיים בבקר ה-VGA

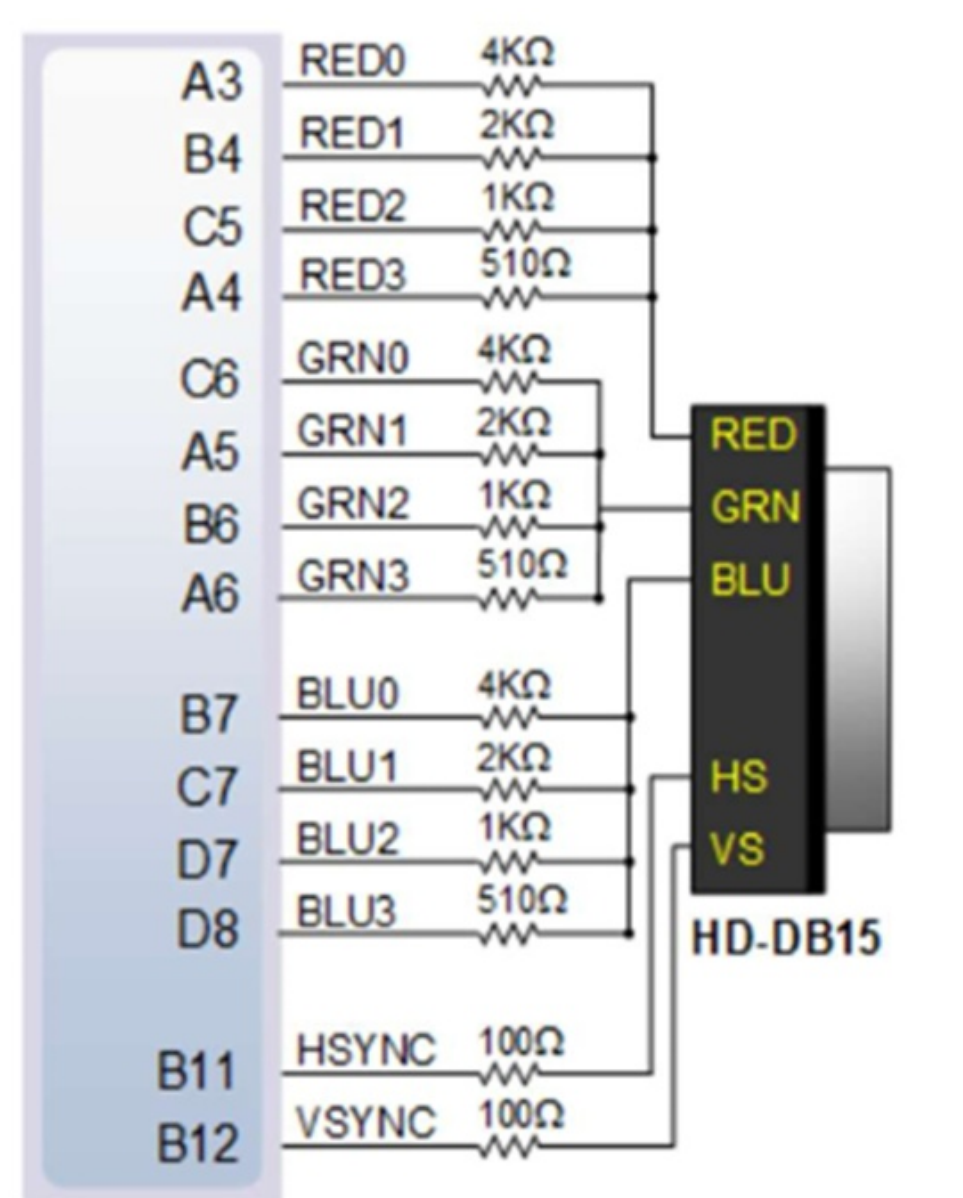
3.2.1 עיבוד הפיקסל והמרה לאנלוגית (DAC)

לאחר שליפת הנתונים מזיכרון ה-BRAM, המילה הנשלפת בגודל 12 ביט ננעלת באוגר pxl_data_reg ומפוצלת לשלושה ערוצים: אדום (4 ביט), ירוק (4 ביט), וכחול (4 ביט). כאשר דגל התצוגה המושהה (in_display_area_delayed) פעיל, ערכי ה-RGB מנותבים ליציאות דרך סינון דיגיטלי (Multiplexing).

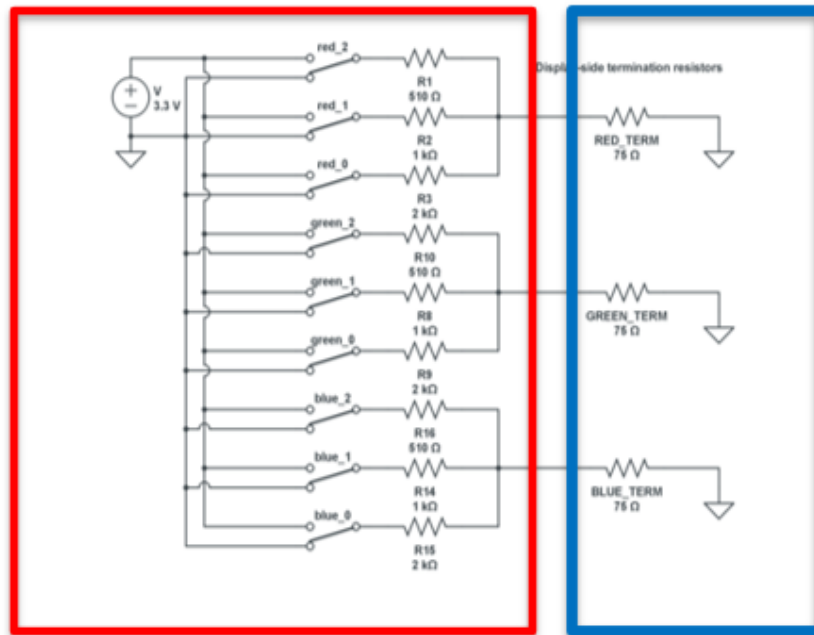


איור 5: תרשים זרימה של עיבוד הפיקסל: מזיכרון ה-BRAM, דרך מנגנון הרישום והסנכרון, ועד לניתוב לערוצי ה-RGB

מאחר שמסך ה-VGA דורש אותות אנלוגיים רציפים, הוטמעה המרה דיגיטלית-לאנלוגית (DAC) פסיבית באמצעות רשת נגדים (Resistor Network). הביט המשמעותי ביותר (MSB) מחובר לנגד בעל הערך הנמוך ביותר (510Ω) להזרמת זרם מרבי, והביט הפחות משמעותי (LSB) מחובר לנגד הגדול ביותר ($4k\Omega$). הזרמים מתנקזים לאדמה דרך סיומת של 75Ω המובנה במסך. שילוב זה יוצר מחלק מתח דינמי הממיר בצורה חלקה את רמות המתח הספרתיות למתח אנלוגי תקני. במקביל, אותות הסנכרון (HSYNC ו-VSYNC) מוגנים מזרם יתר באמצעות נגדי טור של 100Ω .



איור 6: סכמה חשמלית של רשת הנגדים (DAC) ומחבר ה-HD-DB15

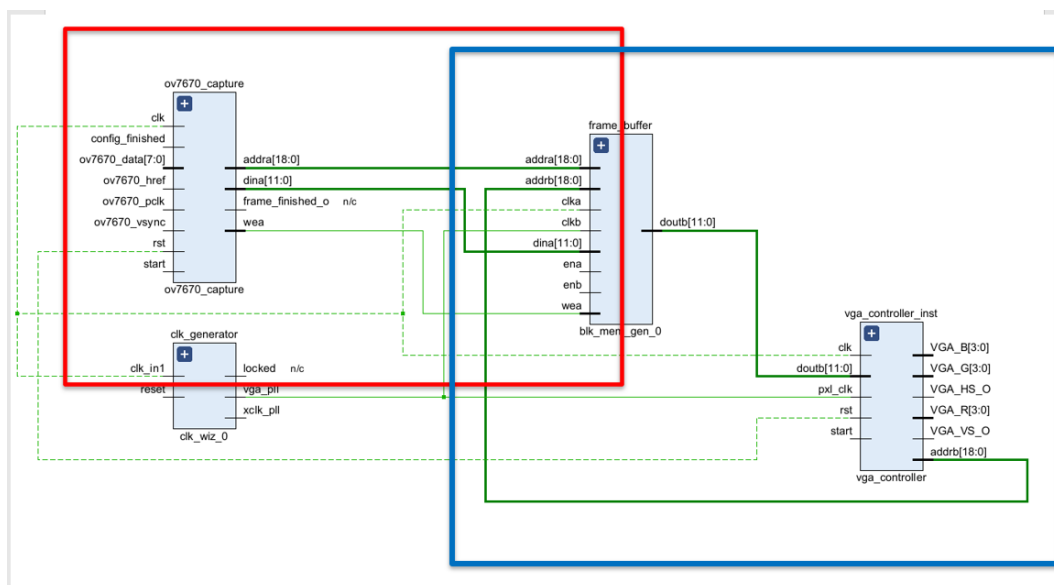


איור 7: תרשים חשמלי המציג את סולם הנגדים בכרטיס (משמאל) אל מול נגד הסיומת במסך (מימין)

3.3 מבט-על: חלוקת המערכת, ארכיטקטורת זיכרון ותחומי שרון

3.3.1 מבט-על על המערכת וזרימת הנתונים

כדי להבין את אופן פעולת המערכת בשלמותה, נבחן את ארכיטקטורת זרימת הנתונים והחלוקה הלוגית של רכיבי החומרה. איור 8 מציג מבט-על הממחיש את שלבי העיבוד השונים ואת גבולות הגזרה.



איור 8: מבט-על של המערכת: חלוקה לליבות עיבוד, תקשורת קליטה (אדום) ותקשורת תצוגה (כחול)

כפי שמשתקף באיור, המערכת נשענת על ארבעה עוגנים מרכזיים:

מחולל שעונים (clk_generator):

רכיב PLL המקבל שעון מערכת של 100MHz ומפיק ממנו שני שעונים נפרדים: שעון 25MHz עבור בקר ה-VGA, ושעון 24MHz ייעודי עבור המצלמה.

זיכרון התמונה (BRAM):

זיכרון א-סינכרוני בתצורת Dual-Port, הפועל כחוצץ בטוח המגשר בין קצב הכתיבה לבין קצב הקריאה. תצורה זו מאפשרת מעבר נתונים יציב ואמין בין תחומי השעון השונים.

התחום האדום (תקשורת המצלמה):

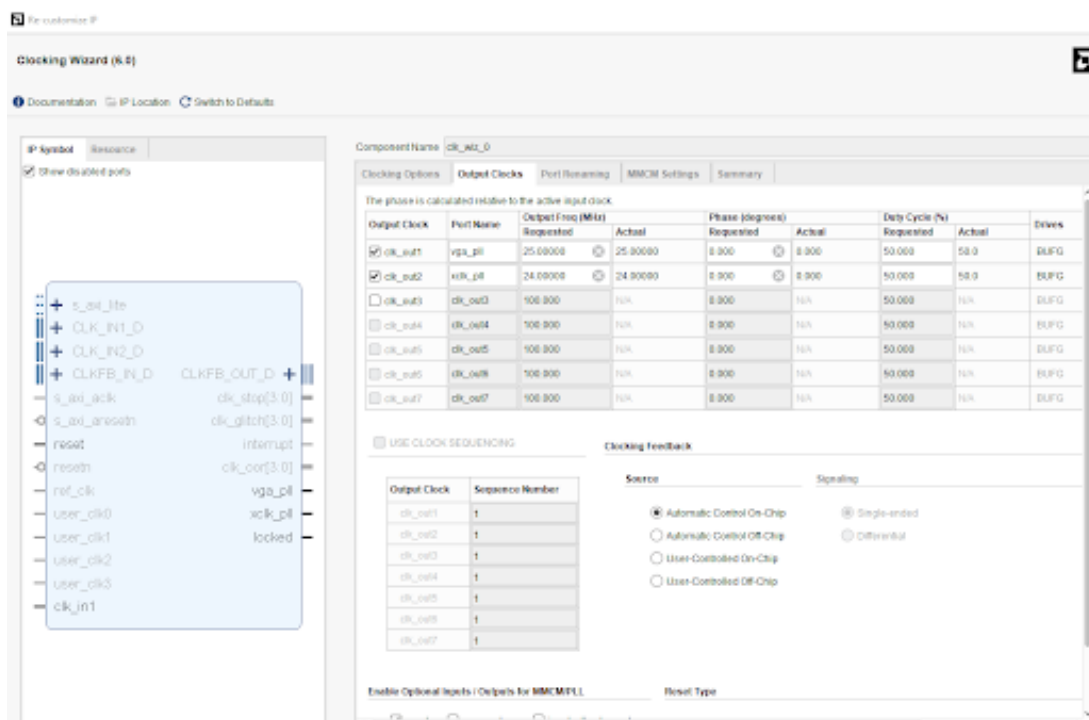
מייצג את התקשורת שבין המצלמה לזיכרון. בחלק זה, לוגיקת הקליטה (ov7670_capture) דוגמת את הפיקסלים ודוחפת אותם פנימה אל פורט A של הזיכרון (clk_a).

התחום הכחול (תקשורת התצוגה - עיקר הדגש):

מייצג את התקשורת שבין הזיכרון לבין המסך. תחום זה (VGA Domain) כולל את שלילת הפיקסלים מפורט B של הזיכרון (clk_b) על ידי בקר ה-VGA וייצור אותות התצוגה. עבודת אימות התכן והסנכרון של מסלול זה מהווה את ליבת המיקוד שלנו בדוח זה.

3.3.2 מחולל השעונים (Clock Generator)

כפי שתואר במבט-העל, באמצעות רכיב ה-Clocking Wizard ב-Vivado, שעון המערכת הראשי של כרטיס ה-Artix-7 (100MHz) מומר לשני שעונים נפרדים ויציבים לחיוניים למערכת:

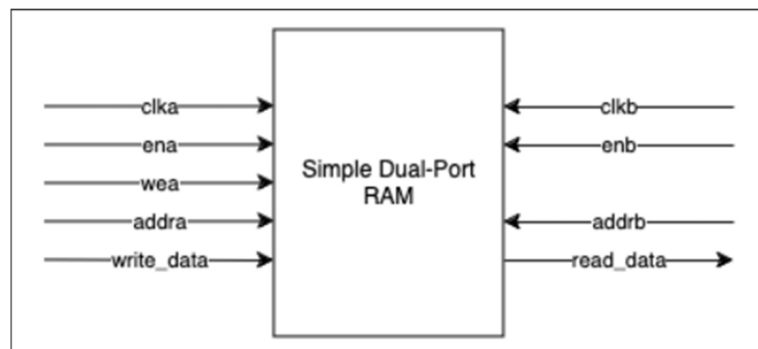


איור 9: ממשק הקינפוג של ה-Clocking Wizard ב-Vivado להפקת שעוני התצוגה והמצלמה

3.3.3 זיכרון התמונה (BRAM Frame Buffer)

רכיב הזיכרון מומש כ-Simple Dual-Port BRAM בגודל $12 \times 307,200$ ביט (נפח המכיל במדויק פריים וידאו שלם ברזולוציית 480×640 בעומק של 12 ביט צבע). תצורה חומרתית זו מספקת הפרדה

מוחלטת לפתרון בעיית חציית תחומי השעון (CDC): פורט A (הכתיבה) קולט את הנתונים מהמצלמה בקצב א-סינכרוני המבוסס על שעון המערכת, בעוד שפורט B (הקריאה) משחרר את הנתונים לבקר ה-VGA בקצב יציב ונפרד של 25 MHz.



איור 10: תרשים בלוקים של זיכרון ה-BRAM בתצורת Dual-Port המאפשרת הפרדת תחומי שעון

3.3.4 ממשק הזיכרון וסנכרון נתונים למסך

בקר התצוגה עובד במונחים של מרחב דו-ממדי (צירים X ו-Y), אך הזיכרון הוא רציף וחד-ממדי. לפיכך, הכתובת לזיכרון מחושבת בזמן אמת על ידי הלוגיקה בהתאם לנוסחה: $Address = X + 640 \times Y$. כדי להתגבר על זמן השהיית הקריאה המובנה ברכיב הזיכרון (Read Latency), הנתון הנשלף (doutb) ננעל בתוך אוגר צינור נתונים (pxl_data_reg), ובמקביל דגל התצוגה מושהה במחזור שעון אחד (in_display_area_delayed). מנגנון זה שומר על סנכרון מושלם בין הנתון המוצג לבין מיקומה הפיזי של הקרן על המסך.

```

-- =====
-- Address Generation
-- באורך 19 ביט BRAM-לכתובת ליניארית ב (X,Y) ממיר את קואורדינטות המסך
-- =====
PROCESS(hsync_reg, vsync_reg, frame_finished, start)
BEGIN
    -- איפוס הכתובת בסוף פריים (חזרה לתחילת המסך/זיכרון)
    IF frame_finished = '1' THEN
        bram_address_next <= (OTHERS => '0');

    -- כאשר המערכת פועלת
    ELSIF start = '1' THEN
        -- שומר סף: בדיקה האם הקרן נמצאת כעת בתוך האזור הפעיל לתצוגה (Active Area)
        IF hsync_reg >= H_DISPLAY_START AND hsync_reg < (H_DISPLAY_START + FRAME_WIDTH) AND
            vsync_reg >= V_DISPLAY_START AND vsync_reg < (V_DISPLAY_START + FRAME_HEIGHT) THEN

            -- לכתובת ליניארית (X,Y) תרגום מתמטי של הקואורדינטות
            bram_address_next <= to_unsigned(((vsync_reg - V_DISPLAY_START) * FRAME_WIDTH) +
            (hsync_reg - H_DISPLAY_START), 19);

            -- שומרים על הכתובת הנוכחית - Blanking/Porch) מחוץ לאזור הפעיל
        ELSE
            bram_address_next <= bram_address_reg;
        END IF;

    ELSE
        bram_address_next <= bram_address_reg;
    END IF;
END PROCESS;

-- =====
-- Pipeline Register
-- גזעל את הפיקסל הנכנס BRAM (Read Latency) מפיצה על זמן התגובה של ה
-- =====
PROCESS (pxl_clk)
BEGIN
    -- דגימה סינכרונית לפי שעון הפיקסלים (25MHz)
    IF rising_edge(pxl_clk) THEN
        IF start = '1' THEN
            -- של הזיכרון B רישום 12 ביטי הצבע שחוזרים מפורט
            pxl_data_reg <= doutb;
        END IF;
    END IF;
END PROCESS;

```

איור 11: קטע מקוד ה-VHDL המציג את תרגום הקואורדינטות בזמן אמת ואת מנגנון ה-Pipeline Register

4 תוצאות

4.1 אימות לוגי ModelSim

תהליך האימות של המערכת חולק לשני חלקים עיקריים. החלק הראשון, שיוצג להלן, מתמקד בתקשורת הפנימית ובסנכרון הנתונים בין בקר ה-VGA לבין זיכרון התמונה (BRAM). החלק השני מוכיח את העמידה בחוקיות הפרוטוקול הפיזי של המסך (תזמוני ה-Blanking וה-Porch).

4.1.1 אימות מסלול הנתונים והסנכרון: VGA Controller ↔ BRAM

בסימולציה זו נבחן מנגנון שלילת הפיקסלים מהזיכרון וניתובם לערוצי הצבע, תוך התמודדות עם אתגרי תזמון חומרתיים:

מניעת גליצים (Glitches) ורעשים:

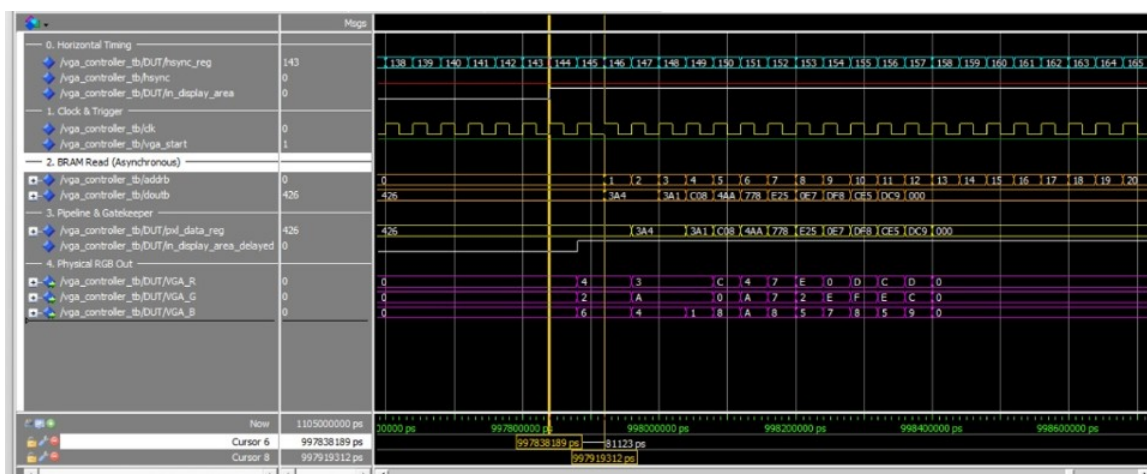
קריאה ישירה מזיכרון ללא תזמון מדויק עלולה לייצר רעשים במוצא בעת החלפת הכתובת. מיושם רגיסטר דוגם (pxl_data_reg) בתוך הבקר, הקולט את הנתון הנכנס doutb ומתעדכן אך ורק בעליית שעון, מה שמבטיח מידע יציב ונקי.

השהיית ה-Pipeline והזרקת הנתונים:

ברגע שדגל אזור התצוגה המעודכן (in_display_area_delayed) עולה לערך לוגי '1', הנתון שהמתין ב-doutb ממחזור השעון הקודם, מנותב באופן מיידי לערוצי ה-RGB הפיזיים.

רציפות זרימת הנתונים:

לאורך האזור הפעיל כולו, בכל מחזור שעון נשלחת בקשת כתובת חדשה (addrb), פיקסל חדש נשלף מהזיכרון, מתייצב דרך הרגיסטר, ומשודר החוצה.

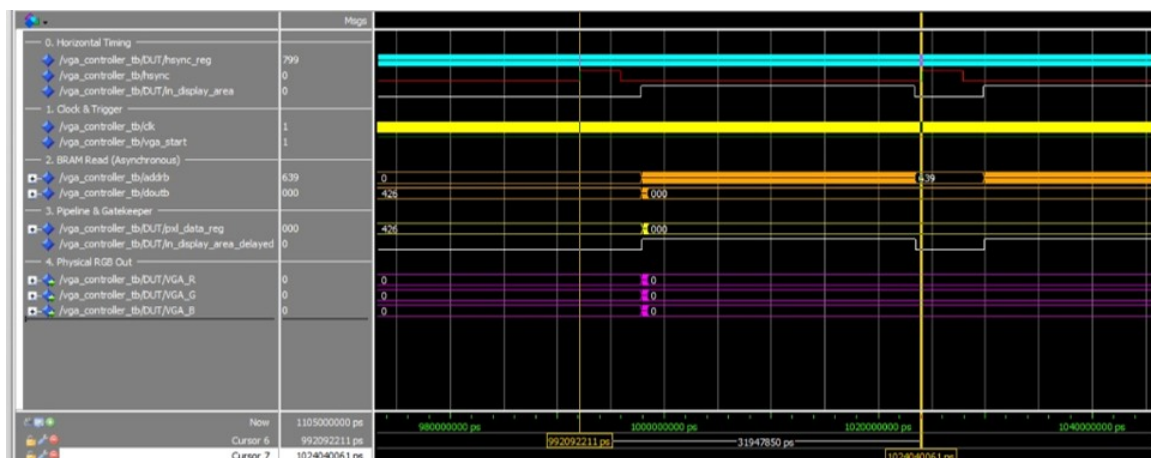


איור 12: סימולציית ModelSim המאמתת את מסלול הנתונים הפנימי: ייצוב הנתון הנשלף מה-BRAM על ידי רגיסטר הדגימה, והעברתו לערוצי ה-RGB

4.1.2 אימות חוקיות הפרוטוקול - מבט-על על מחזור שורה (T_s)

בחלקה השני של סימולציית בקר ה-VGA, מתבצע אימות קפדני של עמידות הלוגיקה בדרישות התזמון הפיזיקליות. המערכת מתוכננת לרזולוציה של 640x480 בקצב רענון 60Hz, המונעת על ידי שעון 25MHz. בהתאם לחישובים התיאורטיים, שורה שלמה אמורה לארוך בדיוק 800 מחזורי שעון.

המונה האופקי (hsync_reg) מבצע ספירה מדויקת ורציפה החל מ-0 ועד לערך המקסימלי של 799, ורק אז מתאפס. מדידת הזמן מאשרת כי משך הזמן תואם במדויק את הדרישה החומרית של התקן.



איור 13: מבט-על של סימולציית מחזור שורה שלם (T_s): אימות ספירת המונה האופקי מ-0 עד 799

4.1.3 אימות דופק הסנכרון האופקי (T_{pw})

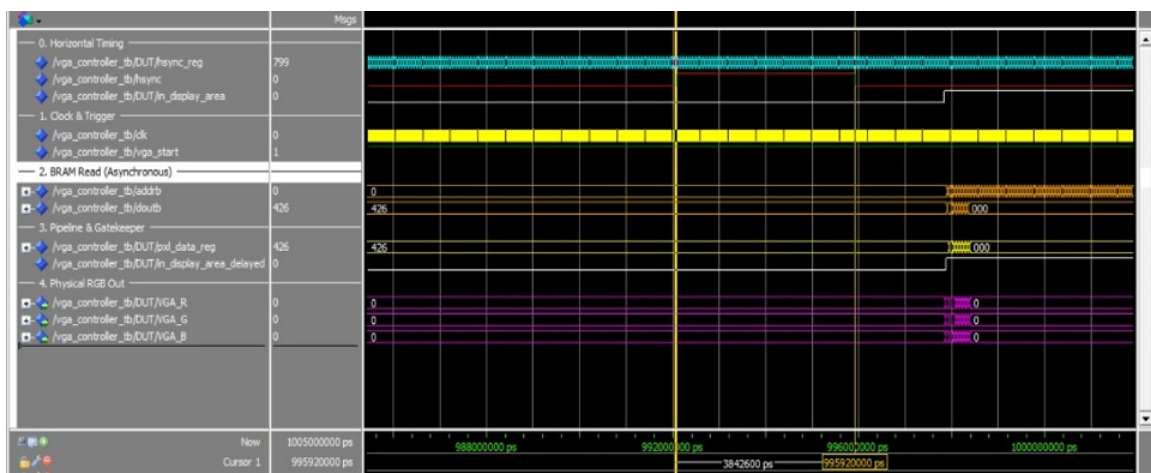
תחילתו של כל מחזור שורה מוגדרת על ידי דופק הסנכרון (Sync Pulse), המסומן בפרוטוקול כ- T_{pw} , ומהווה את הטריגר הפיזי למסך לבצע את חזרת הקרן.

יצירת פולס הסנכרון:

המונה האופקי (hsync_reg) מתחיל את ספירתו מ-0. כל עוד המונה בתחום שבין 0 ל-95 (96 מחזורי שעון), האות hsync עולה ל-'1' לוגי.

אכיפת השחרה (Blanking):

לאורך כל משך פולס הסנכרון, המערכת משתיקה את ערוצי הצבע לערך "0000" (שחור מוחלט), כדי למנוע ציור נתונים שגויים בזמן חזרת הקרן (Retrace).



איור 14: התמקדות בדופק הסנכרון האופקי (T_{pw}): המונה רץ מ-0 עד 95, פולס ה-Sync מופעל, וערוצי הצבע מושתיקים

4.1.4 אימות מרווח ההתייבבות בתחילת שורה (T_{bp} - Back Porch)

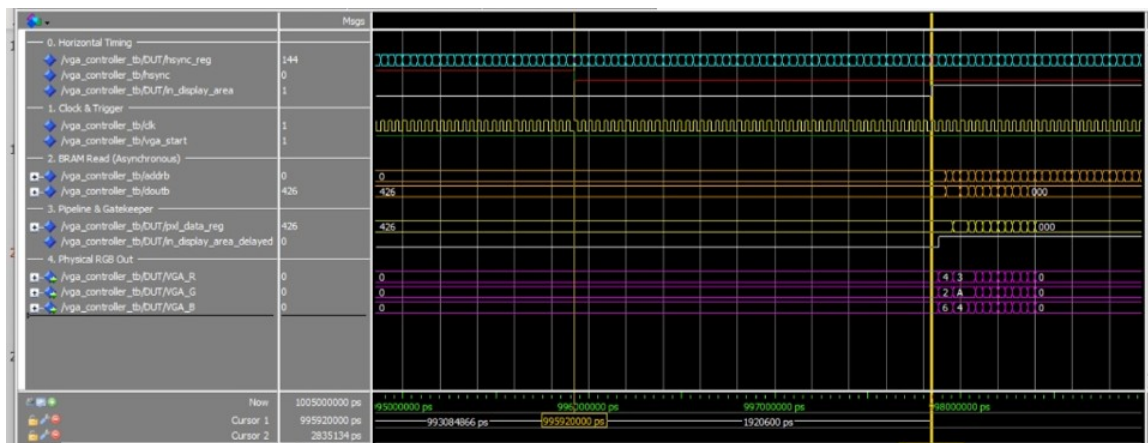
כרונולוגית, מיד לאחר סיום דופק הסנכרון מתחיל שלב ה-Back Porch שמיועד לספק לקרן האלקטרוניים הפיזית "זמן התייבבות" בצד השמאלי של המסך.

השתקה מצטברת:

הפרוטוקול מגדיר 48 מחזורי שעון נוספים עבור ה-Back Porch. בסך הכל המערכת מושתקת מתחילת השורה למשך 144 מחזורי שעון (96+48).

פתיחת חלון הנתונים:

מיד לאחר שהמונה האופקי מסיים את ספירת 144 המחזורים, תהליך ההשתקה מסתיים והנתון הראשון בשורה מנותב החוצה.



איור 15: אימות שלב ה-Back Porch: השתקת ערוצי הצבע לאורך 144 המחזורים הראשונים של השורה

4.1.5 אימות האזור הפעיל (T_{disp} - Active Video)

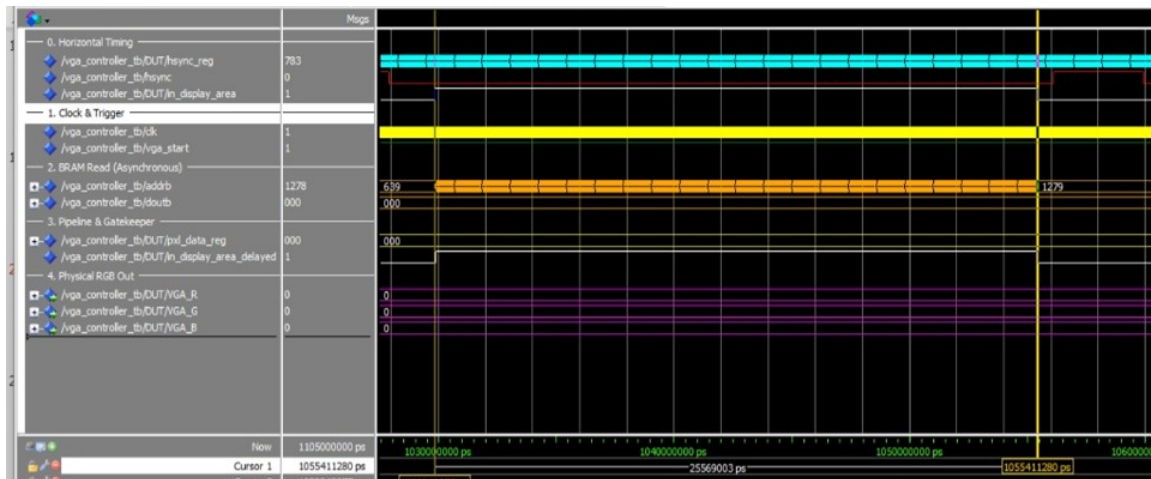
זהו ליבת המחזור האופקי בו הקרן מציירת את המידע הוויזואלי.

תחום המנייה והכתובות:

המונה האופקי סופר מהערך 144 ועד ל-783, המהווים 640 מחזורי שעון פעילים. לאורך חלון זמן זה, אפיק הכתובת לזיכרון (addrb) עובד ומתקדם כראוי.

הערת אימות (Testbench):

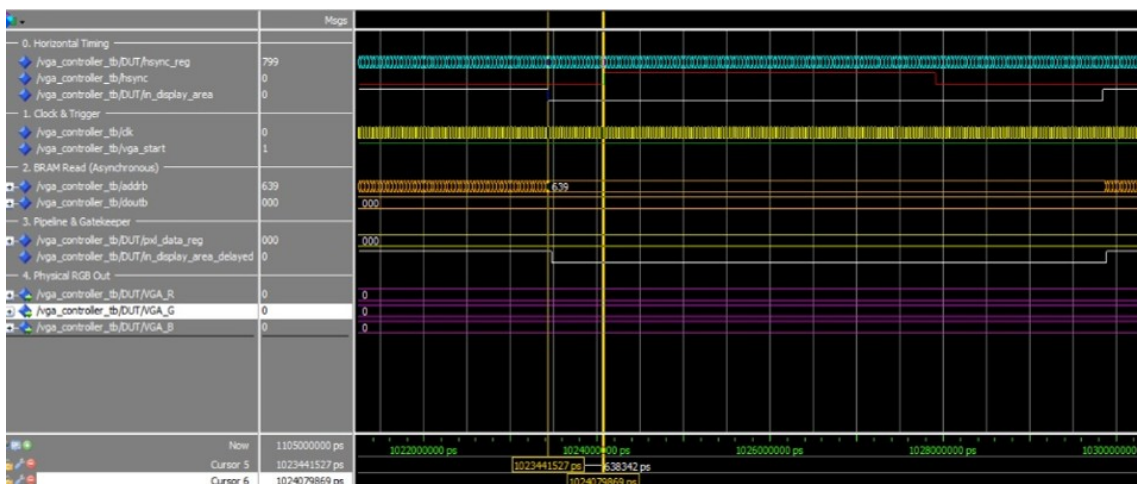
בגרף הסימולציה ערוץ הנתונים ויציאות ה-RGB מראים ערך "0000". בניגוד למצבי ה-Blanking הקודמים, דגל התצוגה במצב פעיל ('1'). היעדר צבע נובע אך ורק מווקטורי הבדיקה של ה-Testbench (זיכרון מאותחל לאפסים) ולא מחסימה לוגית.



איור 16: אימות האזור הפעיל (T_{disp}): מנייה של 640 מחזורים (783-144) ופעילות רציפה של אפיק הכתובת

4.1.6 אימות סיום השורה ומרווח הביטחון (T_{fp} - Front Porch)

השלב החותם את מחזור השורה, המסומן כ- T_{fp} , משמש כ"שולי ביטחון" (Margin) פיזיקליים לפני הטריגר של השורה הבאה. מיד לאחר 640 הפיקסלים הפעילים, המונה האופקי ממשיך לספור מ-784 ועד 799 (16 מחזורי שעון). דגל התצוגה יורד ל-'0', וערוצי ה-RGB מאולצים חזרה ל-'0000' למניעת "מריחות קצה" רגע לפני חזרת הקרן. במחזור העוקב, המונה מתאפס ונסגר מעגל הסריקה הקווית בשלמותו.



איור 17: אימות ה-Front Porch: 16 מחזורי שעון (799-784) של השתקת נתונים בסוף השורה

4.2 אימות חומרה פיזי ILA

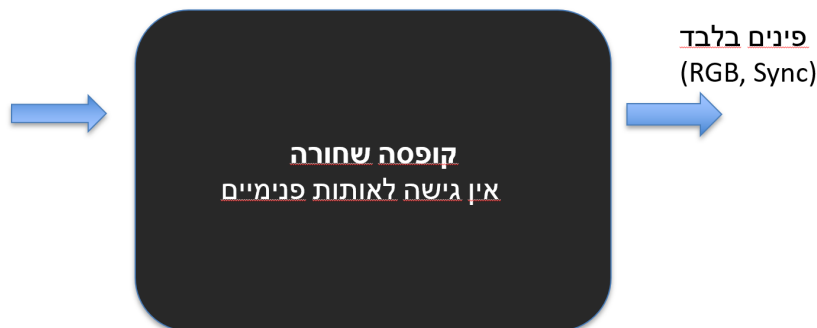
4.2.1 מעבר מתיאוריה לסיליקון

בעוד שסביבת ModelSim (Zero Latency) מאפשרת שקיפות מלאה לכל אות, המעבר למימוש פיזי על גבי כרטיס ה-Artix-7 כפוף לפיזיקה של הרכיב ולהשהיות התפשטות (Propagation Delays). מנקודת

המבט של בוחן חיצוני, ה-FPGA מתנהג כ"קופסה שחורה" ללא אפשרות לראות את האותות הפנימיים (מתי מתאפסים המונים או אילו כתובות נשלחות ל-BRAM).

כדי להתגבר על מגבלה זו ולדגום את האותות בזמן אמת באמצעות מערכת ה-ILA, נדרשנו להכין את התשתית כבר ברמת קוד המקור (VHDL). מאחר שכלי הסינתזה (Synthesis) של Vivado מייעל את התכנון ונוטה להעלים אותות פנימיים שאינם מנותבים לפינים פיזיים, "השתלנו" בקוד תכונות ייעודיות מסוג mark_debug.

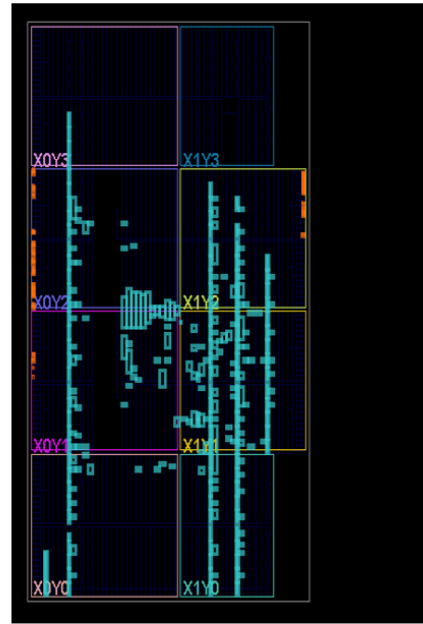
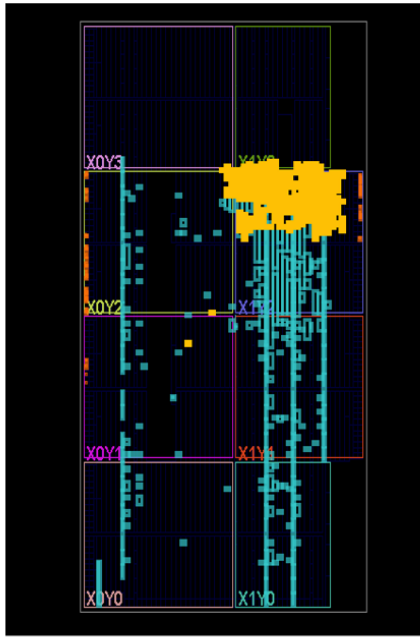
הגדרת השתלים בקוד (לדוגמה: ATTRIBUTE mark_debug OF pxl_data_reg : SIGNAL (IS "true"; אילצה את תוכנת הסינתזה לשמר את האותות הקריטיים כגון אפיק הכתובות, אוגר נתוני הפיקסלים, ומוני התצוגה הפנימיים. פעולה זו למעשה הפכה את המערכת ל"קופסה לבנה" (White Box Testing) ואפשרה לחבר אליה פינים וירטואליים (Probes) ישירות אל תוך הסיליקון.



איור 18: המחשת מגבלת הדיבוג החיצוני והצורך בפתיחת "הקופסה השחורה" באמצעות שתלי ILA

4.2.2 עלות הדיבוג וניתוח Trade-offs בשימוש ב-ILA

כדי להשיג שקיפות נתונים, הוטמע במערכת רכיב Integrated Logic Analyzer (ILA). מדובר בחומרה פיזית הצורכת משאבים רבים ואינה מיועדת למוצר הסופי. השוואה מדויקת של דוחות ניצול המשאבים (Utilization) ב-Vivado חושפת כי בעוד בקר ה-VGA המקורי צרך רק 1,014 יחידות לוגיות, הוספת מערכת הדיבוג (u_ila_0 ו-dbg_hub) דרשה 1,732 יחידות לוגיות נוספות. עובדה זו ממחישה שחומרת הדיבוג עולה למעלה מ-100% ביחס למחיר החומרתי של המערכת התפקודית עצמה.



איור 19: מפת המשאבים על גבי הסיליקון: מימין המערכת ללא ILA, ומשמאל תוספת החומרה עבור מערכת הדיבוג

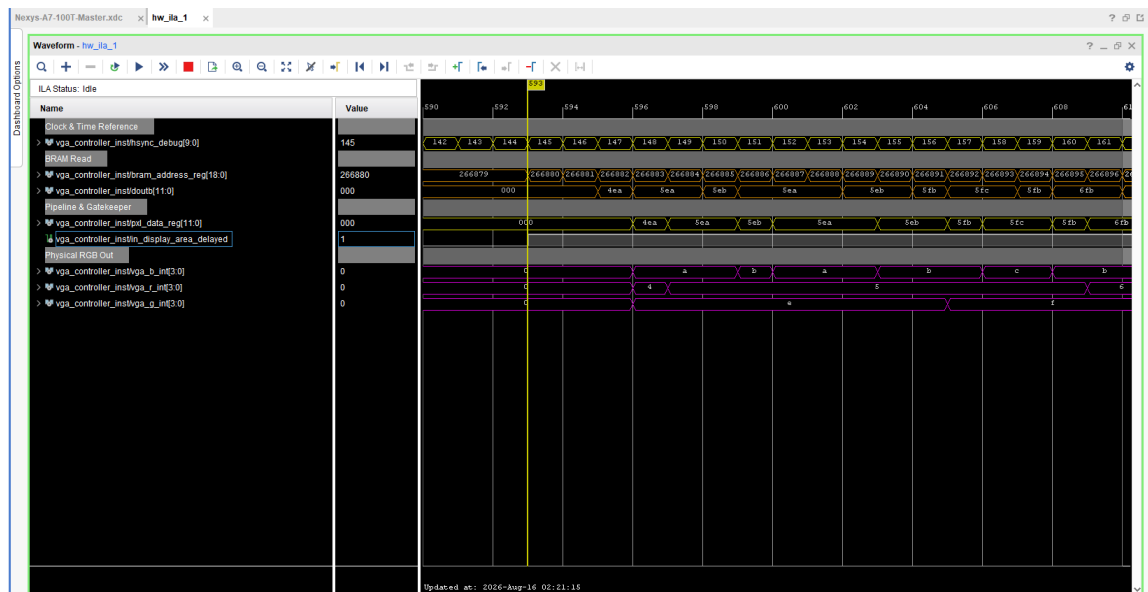
Design Runs											
Utilization											
Hierarchy											
Name	Slice LUTs (63400)	Slice Registers (126800)	F7 Muxes (31700)	F8 Muxes (15850)	Slice (15850)	LUT as Logic (63400)	Block RAM Tile (135)	DSPs (240)	Bonded IOB (210)	BUFCTRL (32)	MMCME2_ADV (6)
top	1014	244	210	97	454	1014	103.5	3	37	4	1
frame_buffer (blk_mem_gen_0)	459	14	93	36	227	459	103.5	0	0	0	0
ov7670_capture (ov7670_capture)	66	78	0	0	37	66	0	1	0	0	0
ov7670_configuration (ov7670_configuration)	421	77	117	61	158	421	0	0	0	0	0
vga_controller_inst (vga_controller)	57	52	0	0	34	57	0	2	0	0	0

Design Runs											
Utilization											
Hierarchy											
Name	Slice LUTs (63400)	Slice Registers (126800)	F7 Muxes (31700)	F8 Muxes (15850)	Slice (15850)	LUT as Logic (63400)	LUT as Memory (19000)	Block RAM Tile (135)	DSPs (240)	Bonded IOB (210)	BUFCTRL (32)
top	2756	2890	258	113	1289	2561	195	105	3	37	5
dog_hub (dog_hub)	449	741	0	0	238	425	24	0	0	0	1
frame_buffer (blk_mem_gen_0)	479	18	93	36	232	479	0	103.5	0	0	0
ov7670_capture (ov7670_capture)	66	78	0	0	37	66	0	0	1	0	0
ov7670_configuration (ov7670_configuration)	421	77	117	61	158	421	0	0	0	0	0
u_ila_0 (u_ila_0)	1283	1901	48	16	628	1112	171	1.5	0	0	0
vga_controller_inst (vga_controller)	47	52	0	0	26	47	0	0	2	0	0

איור 20: השוואת דוחות Utilization ב-Vivado המציגים את תקורת ה-ILA

4.2.3 אימות מסלול הנתונים בזמן אמת (Post-Silicon Validation)

דגימת האמת מתוך החומרה הפיזית מאמתת את זרימת הנתונים מול סביבת ה-ModelSim ביחס של 1:1. הדגימה מוכיחה כי ברגע שדגל אזור התצוגה המושהה עולה, הנתונים המגיעים מ-doutb מוזנים לאחר התייצבות אל תוך px1_data_reg, ומנותבים ישירות לערוצי ה-RGB. בנוסף, מחוץ לאזור הפעיל אפיק הכתובות נותר קפוא, דבר המייעל את פעולת המערכת ומונע קריאות שווא.



איור 21: דגימת ILA מתוך הסיליקון המאמתת את זרימת הנתונים בזמן אמת

5 דיונים

5.1 ניתוח בעיות סנכרון

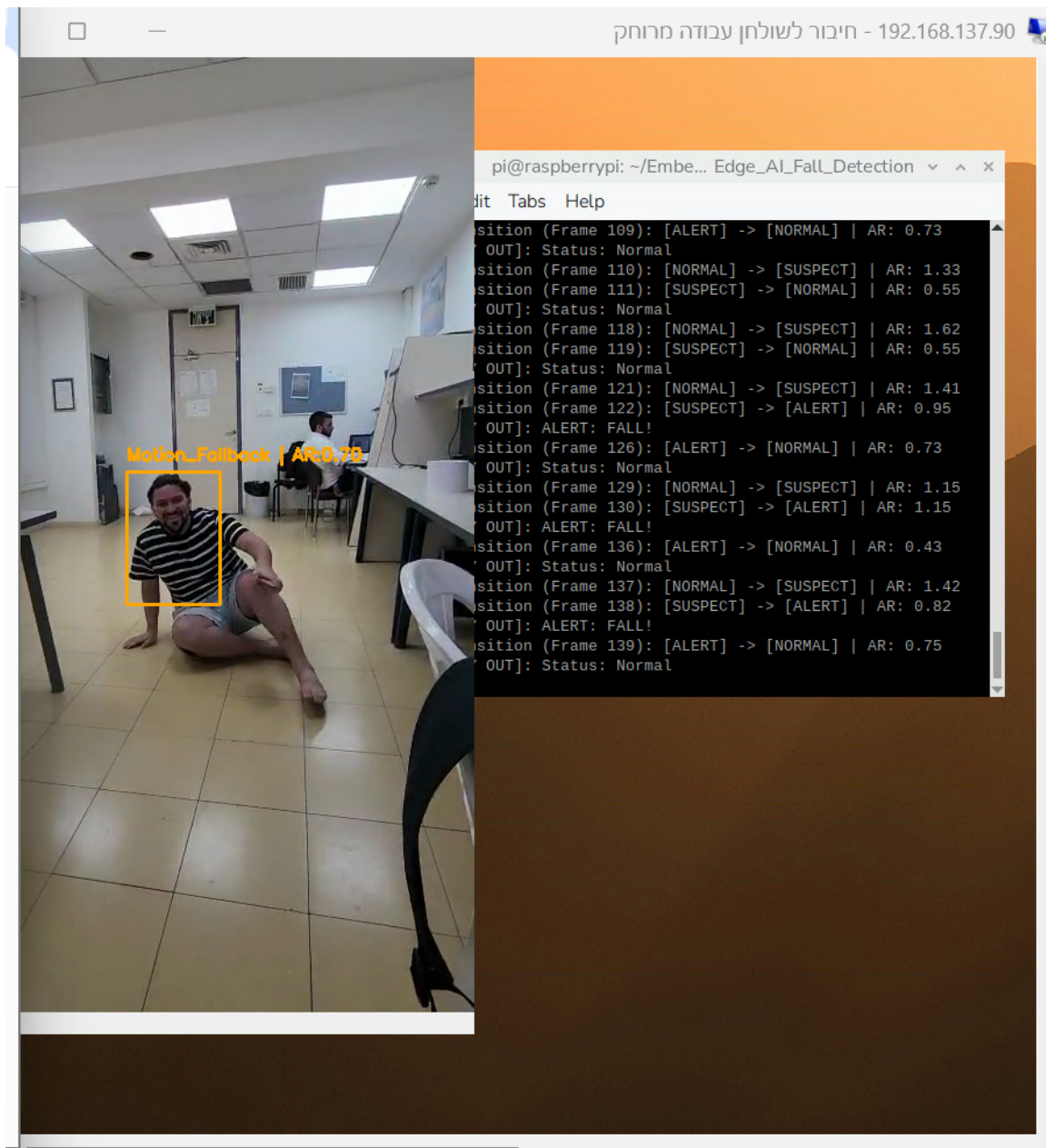
אחד האתגרים ההנדסיים המרכזיים שנותרו לאורך הפרויקט הוא העברת המידע בין שני תחומי שעות א-סינכרוניים (CDC). המצלמה מזינה את המערכת בקצב שעות מהיר של 100MHz, בעוד תותח האלקטרוניים דורש את נתוני הווידאו בקצב איטי יותר של 25MHz. העברה ישירה של נתונים בין שני תחומים אלו עלולה להוביל לבעיות סנכרון חמורות כגון אובדן מידע, מטא-סטביליות (Metastability), וגליצים בתצוגה.

הפתרון הארכיטקטוני שנבחר ונבדק כלל שימוש בזיכרון BRAM בתצורת Simple Dual-Port. תצורה זו מייצרת הפרדה חומרית הרמטית: הכתיבה מתבצעת לפורט A בתדר קליטה, והקריאה מפורט B בתדר שידור. עם זאת, שליפה מזיכרון כרוכה בזמן השהייה מובנה (Read Latency). כדי לפתור דילמה זו מבלי לאבד את סנכרון הפיקסלים, הוכנס מנגנון Pipeline ייעודי: הנתון ננעל בתוך אוגר מקשר (pxl_data_reg), במקביל להשהיית דגל אזור התצוגה במחזור שעות אחד. כפי שהוכח בשלב הבדיקות, תכנון זה יצר מסלול נתונים נקי ומוגן משגיאות תזמון, גם בתנאי אמת על הסיליקון.

5.2 חומרה ייעודית (FPGA) ושילוב מודלי בינה מלאכותית (Edge AI)

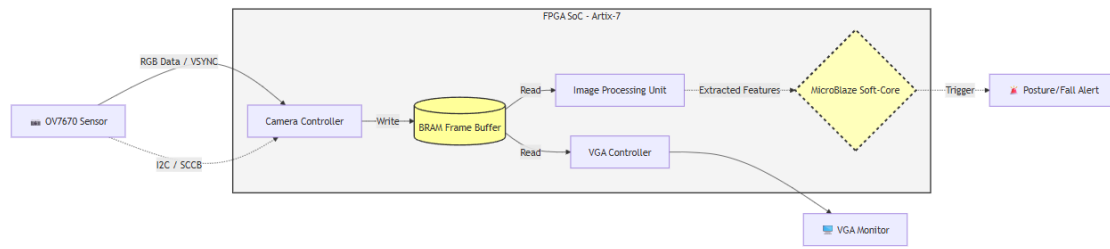
במהלך אפיון המערכת הכוללת (לאחר הכנסת החלק של עיבוד התמונה של אלעד), עדיין נשארו עם פער במנגנון הליבה שאחראי על זיהוי הסכנה. המערכת הגולמית מבוססת על השוואת פיקסלים מבוססת חומרה, המזהה תנועה בצורה מהירה אך נעדרת יכולת להבדיל בין אדם נופל לעצם דומם שנופל, מה שגורם להתראות שווא רבות.

במטרה לפתור זאת, בחנו את שילובה של טכנולוגיית Edge AI. פותחה הוכחת היתכנות (PoC) בסביבת תוכנה (Python ו-Raspberry Pi) אשר הריצה מודל למידה עמוקה (Lightweight Deep Learning) המתמקד בזיהוי אדם ומבנה גופו (Human Detection), בשילוב מנוע מצבים לוגי מתקדם המתריע על נפילה. הניסוי המחיש את העליונות של הפתרון התוכנתי בהורדת שיעור התראות השווא.



איור 22: הוכחת היתכנות על גבי Raspberry Pi: זיהוי נפילה בזמן אמת ומעקב אחר מצבי ההתראה

כיוון הפיתוח העתידי המוצע, יתבסס על הטמעת מעבד מסוג MicroBlaze (Soft-Core) על גבי סיליקון ה-FPGA. חלוקת העבודה תהיה אופטימלית: מודולי ה-RTL הקשיחים (OV7670, BRAM, ו-VGA) ימשיכו לשאת בעומס של ניהול הווידאו בקצב Real-Time ללא קטיעות, בעוד מעבד ה-MicroBlaze יגש לזיכרון ויריץ את מודל ההסקה (AI Inference) מבוסס התוכנה. ארכיטקטורה זו תאפשר הן את מהירות הביצועים של ה-FPGA, ומאפשרת במקביל גמישות ארכיטקטונית מלאה לעדכון הרשת העצבית ושיפור הדיוק באמצעות קוד תוכנה בלבד.



איור 23: הצעת הארכיטקטורה העתידית

6 מסקנות וסיכום

פרויקט זה הציג תהליך פיתוח ואימות מלא של מערכת משובצת לניטור וידאו בזמן אמת. לאורך העבודה, הושם דגש רב על המעבר המורכב שבין תכנון תיאורטי למימוש פיזי אמין, תוך גישור בין העולם הדיגיטלי (RTL) לעולם האנלוגי.

מסקנה הנדסית מרכזית העולה מן העבודה היא חשיבות ההבנה של מורשת החומרה. תכנון בקר ה-VGA דרש התאמה נוקשה לאילוצים פיזיקליים של תותחי אלקטרונים (CRT), מה שחייב ניהול לוגי קפדני של זמני החזרה (Retrace) ואזורי השחרה (Blanking). כמו כן, המרת האותות באמצעות מחלק מתח מבוסס רשת נגדים (DAC) המחישה את הצורך להבין את המעגל החשמלי הפיזי המשלים את קוד ה-VHDL.

מסלול הנתונים הוויזואלי תוכנן ואומת במלואו, תוך התמודדות עם אתגרי חציית תחומי שיעון א-סינכרוניים באמצעות הפרדה חומרית בזיכרון ה-BRAM. מנגנוני ה-Pipeline שהוטמעו הבטיחו שקיפות וסנכרון, ואלו אומתו בצורה כפולה: תחילה בסביבה וירטואלית ואידיאלית (ModelSim), ולאחר מכן באופן פיזי על גבי הסיליקון באמצעות כלי הדיבוג הפנימי (ILA). ההתמודדות עם ה-ILA וההבנה של תקורת המשאבים הכרוכה בו (למעלה מ-100% תוספת חומרה), הוכיחו כי בעולם ה-FPGA ניהול משאבים ויכולת תצפית הם גורמים קריטיים להצלחה.

במקביל לעבודת החומרה הקשיחה, ניתוח מגבלותיה של מערכת זיהוי התנועה הגולמית הוביל למסקנה כי נדרשת שכבת הבנה סמנטית. הצעת הפיתוח העתידית, המבוססת על גישת "תכנון חומרה-תוכנה משולב" (HW/SW Co-Design), הציגה פתרון אלגנטי: שימוש במעבד Soft-Core פנימי מסוג MicroBlaze להרצת מודלי למידה עמוקה (Edge AI), תוך שחרור הלוגיקה המקבילית לניהול רציף של חיישני התמונה והתצוגה.

לסיכום, העבודה משקפת מחזור פיתוח שלם – מפיזיקה ואותות אנלוגיים, דרך סינתזה ואימות קוד RTL, ועד לארכיטקטורת מערכת (System-Level Design) חכמה.

References

- [1] OmniVision Technologies, ``OV7670/OV7171 CMOS VGA (640x480) CameraChip Sensor," Datasheet, 2006.
- [2] Digilent Inc., ``Nexys A7 FPGA Trainer Board Reference Manual," 2019.
- [3] A. G. Howard et al., ``MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications," *arXiv preprint arXiv:1704.04861*, 2017.

Appendix A: Source Code - VGA Controller (VHDL)

```

1  LIBRARY ieee;
2  USE ieee.std_logic_1164.ALL;
3  USE ieee.numeric_std.ALL;
4
5  ENTITY vga_controller IS
6      PORT (
7          clk : IN STD_LOGIC;
8          rst : IN STD_LOGIC;
9          pxl_clk : IN STD_LOGIC;
10         VGA_HS_0 : OUT STD_LOGIC;
11         VGA_VS_0 : OUT STD_LOGIC;
12         VGA_R : OUT STD_LOGIC_VECTOR (3 DOWNTO 0);
13         VGA_B : OUT STD_LOGIC_VECTOR (3 DOWNTO 0);
14         VGA_G : OUT STD_LOGIC_VECTOR (3 DOWNTO 0);
15
16         start : IN STD_LOGIC;
17
18         --frame_buffer signals
19         addrb : OUT STD_LOGIC_VECTOR(18 DOWNTO 0);
20         doutb : IN STD_LOGIC_VECTOR(11 DOWNTO 0) --pixel data
21     );
22 END vga_controller;
23
24 ARCHITECTURE rtl OF vga_controller IS
25     --***640x480@60Hz***-- Requires 25 MHz clock
26     CONSTANT FRAME_WIDTH : NATURAL := 640;
27     CONSTANT FRAME_HEIGHT : NATURAL := 480;
28
29     --      VGA      640x480@60Hz
30     CONSTANT H_SYNC_PULSE_WIDTH : NATURAL := 96; -- Sync pulse
31     CONSTANT H_BACK_PORCH : NATURAL := 48; -- Back porch
32     CONSTANT H_FRONT_PORCH : NATURAL := 16; -- Front porch
33     CONSTANT H_TOTAL_LINE : NATURAL := 800; -- Total = 96+48+640+16
34
35     CONSTANT V_SYNC_PULSE_WIDTH : NATURAL := 2; -- Sync pulse
36     CONSTANT V_BACK_PORCH : NATURAL := 29; -- Back porch
37     CONSTANT V_FRONT_PORCH : NATURAL := 10; -- Front porch
38     CONSTANT V_MAX_LINE : NATURAL := 521; -- Total = 2+33+480+10
39
40     CONSTANT H_POL : STD_LOGIC := '0';
41     CONSTANT V_POL : STD_LOGIC := '0';
42

```

```

43 SIGNAL hsync_reg, hsync_next : INTEGER RANGE 0 TO H_TOTAL_LINE := 0;
44 SIGNAL vsync_reg, vsync_next : INTEGER RANGE 0 TO V_MAX_LINE := 0;
45
46 SIGNAL bram_address_reg, bram_address_next : unsigned(18 DOWNT0 0) := (
OTHERS => '0');
47
48 SIGNAL line_finished : STD_LOGIC := '0';
49 SIGNAL frame_finished : STD_LOGIC := '0';
50
51 --           ): Sync -> Back Porch -> Display)
52 CONSTANT H_DISPLAY_START : NATURAL := H_SYNC_PULSE_WIDTH + H_BACK_PORCH
; -- 96+48=144
53 CONSTANT V_DISPLAY_START : NATURAL := V_SYNC_PULSE_WIDTH + V_BACK_PORCH
; -- 2+33=35
54
55 --
56 SIGNAL display_x : INTEGER RANGE 0 TO FRAME_WIDTH - 1;
57 SIGNAL display_y : INTEGER RANGE 0 TO FRAME_HEIGHT - 1;
58 SIGNAL in_display_area : STD_LOGIC;
59
60 SIGNAL in_display_area_delayed : STD_LOGIC := '0';
61
62 SIGNAL pxl_data_reg : STD_LOGIC_VECTOR(11 DOWNT0 0) := (OTHERS => '0');
63
64 --
65 SIGNAL vga_r_int, vga_g_int, vga_b_int : STD_LOGIC_VECTOR(3 DOWNT0 0);
66
67
68 -- =====
69 -- **** VHDL ILA Probes (White Box Testing) ****
70 -- =====
71
72
73 -- =====
74 -- **** -ILA ****
75 SIGNAL hsync_debug : STD_LOGIC_VECTOR(9 DOWNT0 0);
76 -- =====
77
78
79
80 ATTRIBUTE mark_debug : STRING;
81
82 -- 1.
83 ATTRIBUTE mark_debug OF hsync_debug : SIGNAL IS "true";
84 ATTRIBUTE mark_debug OF in_display_area_delayed : SIGNAL IS "true";
85
86 -- 2.           )Latency & Data Flow)

```

```

87     ATTRIBUTE mark_debug OF bram_address_reg : SIGNAL IS "true";
88     ATTRIBUTE mark_debug OF doutb : SIGNAL IS "true";
89     ATTRIBUTE mark_debug OF pxl_data_reg : SIGNAL IS "true";
90
91     -- 3.
92     ATTRIBUTE mark_debug OF vga_r_int : SIGNAL IS "true";
93     ATTRIBUTE mark_debug OF vga_g_int : SIGNAL IS "true";
94     ATTRIBUTE mark_debug OF vga_b_int : SIGNAL IS "true";
95     -- =====
96
97
98 BEGIN
99
100     -- =====
101     -- ****          ) INTEGER)          ) STD_LOGIC_VECTOR)          -ILA ****
102     hsync_debug <= std_logic_vector(to_unsigned(hsync_reg, 10));
103     -- =====
104
105     --
106     display_x <= hsync_reg - H_DISPLAY_START WHEN hsync_reg >=
H_DISPLAY_START AND hsync_reg < (H_DISPLAY_START + FRAME_WIDTH) ELSE 0;
107     display_y <= vsync_reg - V_DISPLAY_START WHEN vsync_reg >=
V_DISPLAY_START AND vsync_reg < (V_DISPLAY_START + FRAME_HEIGHT) ELSE 0;
108
109     --
110     in_display_area <= '1' WHEN (hsync_reg >= H_DISPLAY_START AND hsync_reg
< (H_DISPLAY_START + FRAME_WIDTH) AND
111                                vsync_reg >= V_DISPLAY_START AND
vsync_reg < (V_DISPLAY_START + FRAME_HEIGHT)) ELSE '0';
112
113     addrb <= STD_LOGIC_VECTOR(bram_address_reg);
114
115     line_finished <= '1' WHEN hsync_reg = H_TOTAL_LINE - 1 ELSE '0';
116     frame_finished <= '1' WHEN vsync_reg = V_MAX_LINE - 1 ELSE '0';
117
118     --      H-Sync (Sync      ,      !)
119     VGA_HS_0 <= NOT H_POL WHEN (hsync_reg >= 0 AND hsync_reg <
H_SYNC_PULSE_WIDTH) ELSE H_POL;
120
121     --      V-Sync (Sync      ,      !)
122     VGA_VS_0 <= NOT V_POL WHEN (vsync_reg >= 0 AND vsync_reg <
V_SYNC_PULSE_WIDTH) ELSE V_POL;
123
124     --
125     hsync_next <= 0 WHEN line_finished = '1' ELSE
hsync_reg + 1 WHEN start = '1' ELSE
126     hsync_reg;
127

```

```

128
129 vsync_next <= 0 WHEN frame_finished = '1' ELSE
130     vsync_reg + 1 WHEN line_finished = '1' ELSE
131     vsync_reg;
132
133 --          BRAM
134 PROCESS(hsync_reg, vsync_reg, frame_finished, start)
135 BEGIN
136     IF frame_finished = '1' THEN
137         bram_address_next <= (OTHERS => '0');
138     ELSIF start = '1' THEN
139         IF hsync_reg >= H_DISPLAY_START AND hsync_reg < (
140             H_DISPLAY_START + FRAME_WIDTH) AND
141             vsync_reg >= V_DISPLAY_START AND vsync_reg < (
142                 V_DISPLAY_START + FRAME_HEIGHT) THEN
143                 bram_address_next <= to_unsigned(((vsync_reg -
144                     V_DISPLAY_START) * FRAME_WIDTH) + (hsync_reg - H_DISPLAY_START), 19);
145             ELSE
146                 bram_address_next <= bram_address_reg;
147             END IF;
148         ELSE
149             bram_address_next <= bram_address_reg;
150         END IF;
151     END PROCESS;
152
153 PROCESS (pxl_clk)
154 BEGIN
155     IF rising_edge(pxl_clk) THEN
156         IF start = '1' THEN
157             pxl_data_reg <= doutb; --          -BRAM
158         END IF;
159     END IF;
160 END PROCESS;
161
162 --          )          -ILA)
163 vga_r_int <= pxl_data_reg(11 DOWNT0 8) WHEN in_display_area_delayed =
164 '1' ELSE "0000";
165 vga_g_int <= pxl_data_reg(7 DOWNT0 4) WHEN in_display_area_delayed =
166 '1' ELSE "0000";
167 vga_b_int <= pxl_data_reg(3 DOWNT0 0) WHEN in_display_area_delayed =
168 '1' ELSE "0000";
169
170 --
171 VGA_R <= vga_r_int;
172 VGA_G <= vga_g_int;
173 VGA_B <= vga_b_int;
174

```

```

169  PROCESS (pxl_clk, rst)
170  BEGIN
171      IF rst = '1' THEN
172          hsync_reg <= 0;
173          vsync_reg <= 0;
174          bram_address_reg <= (OTHERS => '0');
175          in_display_area_delayed <= '0';
176      ELSIF rising_edge(pxl_clk) THEN
177          IF start = '1' THEN
178              hsync_reg <= hsync_next;
179              vsync_reg <= vsync_next;
180              bram_address_reg <= bram_address_next;
181              in_display_area_delayed <= in_display_area;
182          END IF;
183      END IF;
184  END PROCESS;
185
186  END ARCHITECTURE;

```

Listing 1: VGA Controller VHDL Source Code

Appendix B: Source Code - Simulation Verification (Testbench)

```
1  LIBRARY ieee;
2  USE ieee.std_logic_1164.ALL;
3  USE ieee.numeric_std.ALL;
4
5  USE std.textio.ALL;
6  USE std.env.finish;
7
8  ENTITY vga_controller_tb IS
9  END vga_controller_tb;
10
11 ARCHITECTURE sim OF vga_controller_tb IS
12
13     CONSTANT pxclk_hz : INTEGER := 25e6;
14     CONSTANT clk_period : TIME := 1 sec / pxclk_hz;
15
16     SIGNAL clk : STD_LOGIC := '1';
17     SIGNAL rst : STD_LOGIC := '1';
18
19     SIGNAL hsync : STD_LOGIC := '0';
20     SIGNAL vsync : STD_LOGIC := '0';
21     SIGNAL vga_red : STD_LOGIC_VECTOR (3 DOWNTO 0) := (OTHERS => '0');
22     SIGNAL vga_blue : STD_LOGIC_VECTOR (3 DOWNTO 0) := (OTHERS => '0');
23     SIGNAL vga_green : STD_LOGIC_VECTOR (3 DOWNTO 0) := (OTHERS => '0');
24
25     SIGNAL addrb : STD_LOGIC_VECTOR(18 DOWNTO 0) := (OTHERS => '0');
26     SIGNAL doutb : STD_LOGIC_VECTOR(11 DOWNTO 0) := (OTHERS => '0');
27
28     SIGNAL vga_start : STD_LOGIC := '0';
29
30 BEGIN
31
32     doutb <= x"426" WHEN addrb = "000000000000000000" ELSE
33         x"3a4" WHEN addrb = "000000000000000001" ELSE
34         x"3a4" WHEN addrb = "000000000000000010" ELSE
35         x"3a1" WHEN addrb = "000000000000000011" ELSE
36         x"c08" WHEN addrb = "000000000000000100" ELSE
37         x"4aa" WHEN addrb = "000000000000000101" ELSE
38         x"778" WHEN addrb = "000000000000000110" ELSE
39         x"e25" WHEN addrb = "000000000000000111" ELSE
40         x"0e7" WHEN addrb = "000000000000001000" ELSE
41         x"df8" WHEN addrb = "000000000000001001" ELSE
42         x"ce5" WHEN addrb = "000000000000001010" ELSE
43         x"dc9" WHEN addrb = "000000000000001011" ELSE
44         x"376" WHEN addrb = "1001000110001010110" ELSE
45         x"eca" WHEN addrb = "1001000110001010111" ELSE
```

```

46      x"916" WHEN addrb = "1001000110001011000" ELSE
47      x"184" WHEN addrb = "1001000110001011001" ELSE
48      x"7d6" WHEN addrb = "1001000110001011010" ELSE
49      x"f8f" WHEN addrb = "1001000110001011011" ELSE
50      x"85a" WHEN addrb = "1001000110001011100" ELSE
51      x"7d1" WHEN addrb = "1001000110001011101" ELSE
52      x"49b" WHEN addrb = "1001000110001011110" ELSE
53      x"3ad" WHEN addrb = "1001000110001011111" ELSE
54      x"cf7" WHEN addrb = "1001000110001100000" ELSE
55          (OTHERS => '0');
56
57      clk <= NOT clk AFTER clk_period / 2;
58
59      DUT : ENTITY work.vga_controller(rtl)
60          PORT MAP(
61              clk => clk,
62              rst => rst,
63              pxl_clk => clk,
64              VGA_HS_0 => hsync,
65              VGA_VS_0 => vsync,
66              start => vga_start,
67              VGA_R => vga_red,
68              VGA_B => vga_blue,
69              VGA_G => vga_green,
70              addrb => addrb,
71              doutb => doutb
72          );
73
74      vga_start <= '1';
75
76      SEQUENCER_PROC : PROCESS
77      BEGIN
78          WAIT FOR clk_period * 2;
79
80          rst <= '0';
81
82          WAIT FOR clk_period * 10;
83          WAIT ON vsync UNTIL falling_edge(vsync);
84
85          WAIT ON vsync UNTIL rising_edge(vsync);
86
87          WAIT FOR clk_period * 100;
88
89      END PROCESS;
90
91      --
=====

```



```

92  -- Professional Verification Checker: Automatic VGA Standard Assertion
93  --
=====

94  vga_timing_checker : PROCESS
95      VARIABLE hsync_start : TIME := 0 ns;
96      VARIABLE pulse_width : TIME := 0 ns;
97  BEGIN
98      -- 1.
99      WAIT UNTIL rst = '0';
100
101      -- 2.                -HSYNC
102      WAIT UNTIL falling_edge(hsync);
103      hsync_start := NOW;
104
105      WAIT UNTIL rising_edge(hsync);
106      pulse_width := NOW - hsync_start;
107
108      --
=====

109      -- ] [: Assertion Pulse Width ) 20 (
110      --      : )28160 ns) .
111      --      : 25MHz 40ns.
112      --      -HSYNC 96 .
113      --      " : 96 * 40ns = 3840ns.
114      --      : ,
115      --      -RTL VESA -.
116      --
=====

117      IF pulse_width = 3840 ns THEN
118          REPORT ">>> [VERIFICATION PASSED]: HSYNC pulse width is EXACTLY
119          3.84us (96 clocks) as per VGA 640x480@60Hz standard." SEVERITY NOTE;
120      ELSE
121          REPORT ">>> [VERIFICATION ERROR]: HSYNC pulse width mismatch!
122          Measured: " & TIME'IMAGE(pulse_width) SEVERITY ERROR;
123      END IF;
124
125      -- 3. Blanking )) -
126      WAIT UNTIL falling_edge(hsync);
127      WAIT FOR 50 ns; --
128      ASSERT (vga_red = "0000" AND vga_green = "0000" AND vga_blue = "
129      0000")
130      REPORT ">>> [VERIFICATION ERROR]: Color leak detected during
131      HSYNC blanking period!" SEVERITY ERROR;

```

```
128
129     REPORT ">>> [VERIFICATION PASSED]: Blanking enforcement (RGB=0x0
130     during sync) verified successfully." SEVERITY NOTE;
131
132     WAIT; --
133     END PROCESS;
134 END ARCHITECTURE;
```

Listing 2: ModelSim Testbench VHDL Code